REHOSPACE REAL ESTATE PLATFORM (RREP)

MASTER PRODUCT, BUSINESS, TECHNICAL & IMPLEMENTATION SPECIFICATION

Version: 1.0
Product: RehoSpace Real Estate Platform (RREP)
Document Type: Master Product Specification
Owner: RehoSpace
Status: Development Baseline
Classification: Internal / Product Development

DOCUMENT PURPOSE

This Master Specification consolidates the complete product definition of RREP into one authoritative document.

It defines:

- Why RREP exists.
- What RREP does.
- Who RREP serves.
- How RREP is commercialized.
- How RREP is architected.
- How RREP should be developed.
- How RREP should be deployed.
- How RREP should be operated.
- How its user interfaces should be designed.
- How the product will evolve.

This document shall serve as the primary reference for product management, business analysis, software development, quality assurance, deployment, implementation, and future product expansion.

MASTER TABLE OF CONTENTS

VOLUME 0 — PRODUCT VISION & STRATEGY

0.1 Executive Summary

0.2 Product Vision

0.3 Product Mission

0.4 Product Philosophy

0.5 Problem Statement

0.6 Market Opportunity

0.7 Target Customers

0.8 Product Positioning

0.9 Competitive Strategy

0.10 Product Objectives

0.11 Product Principles

0.12 Value Proposition

0.13 Product Ecosystem

0.14 Business Model

0.15 Licensing Strategy

0.16 Product Editions

0.17 Technology Strategy

0.18 Artificial Intelligence Strategy

0.19 Security Strategy

0.20 Product Roadmap

0.21 Go-To-Market Strategy

0.22 Brand Strategy

0.23 Success Metrics

0.24 Long-Term Vision

0.25 Conclusion

VOLUME I — BUSINESS & FUNCTIONAL PRODUCT SPECIFICATION

PART I — PRODUCT FOUNDATION

1. Product Definition

2. Product Scope

3. Product Architecture Concept

4. User Types

5. Roles & Permissions

6. Organization & Branch Structure

7. Configuration & Feature Licensing

PART II — BUSINESS MODULES

BM-001 — Platform Administration

BM-002 — Organization & Branch Management

BM-003 — User, Roles & Access Management

BM-004 — Property & Asset Management

BM-005 — Land & Parcel Management

BM-006 — Survey & GIS Management

BM-007 — Property Listing & Marketplace

BM-008 — Customer Relationship Management

BM-009 — Lead & Sales Management

BM-010 — Reservation & Booking Management

PART III — CROSS-MODULE REQUIREMENTS

VOLUME II — TECHNICAL ARCHITECTURE & DEVELOPMENT BLUEPRINT

31. Technical Objectives

32. Core Architectural Principles

33. Product Deployment Model

34. High-Level System Architecture

35. Application Architecture

36. Module Architecture

37. Shared Services

38. Recommended Technology Stack

39. Backend Architecture

40. Frontend Architecture

41. Mobile Architecture

42. Database Architecture

43. API Architecture

44. Authentication & Authorization

45. File & Document Storage

46. Caching & Queues

47. Search Architecture

48. GIS Architecture

49. Notification Architecture

50. AI Architecture

VOLUME III — IMPLEMENTATION, DEPLOYMENT & OPERATIONS

VOLUME IV — UI/UX DESIGN SYSTEM & PRODUCT STANDARDS

91. Typography

92. Color System

93. Layout System

94. Navigation

95. Forms

96. Tables

97. Dashboards

98. Cards & Components

99. Notifications & Alerts

100. Modals & Dialogues

101. Loading & Empty States

102. Error States

103. Responsive Design

104. Accessibility

105. Mobile UX

106. Customer Portal UX

107. Admin UX

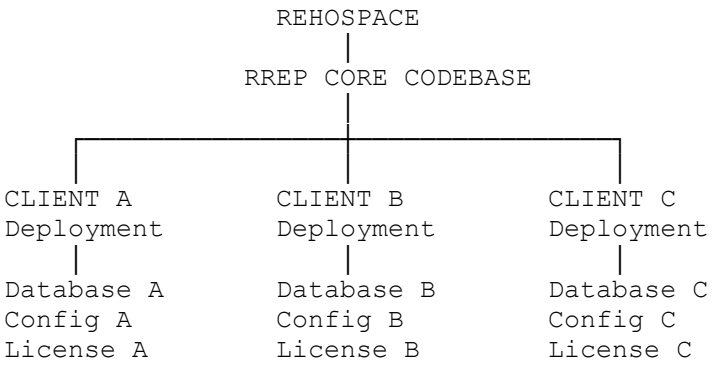
108. Design Consistency Standards

MASTER PRODUCT PRINCIPLE

The entire system shall be built around one fundamental commercial architecture:

RREP is a reusable real estate software product owned and maintained by RehoSpace. Each customer receives a standalone deployment of the same product, independently configured and licensed according to the modules and features purchased.

This means:



There is **no requirement for a centralized multi-tenant SaaS architecture**.

Each customer installation is logically and operationally independent.

RehoSpace controls:

- Product source code
- Product versions
- Licensing
- Available modules
- Feature packages
- Updates
- Support
- Optional integrations
- AI services
- Professional services

The customer controls its own:

- Organization data
- Users
- Properties
- Transactions
- Configuration
- Business operations

TECHNICAL BASELINE

The initial implementation should use a practical, maintainable stack:

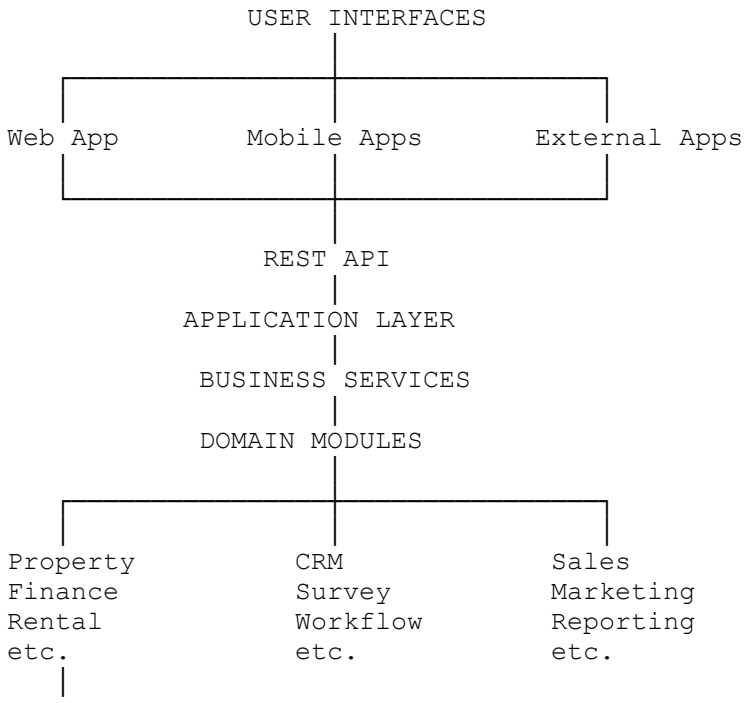
Layer	Recommended Technology
Backend	Laravel / PHP
Database	MySQL
Frontend	Bootstrap 5
JavaScript	jQuery

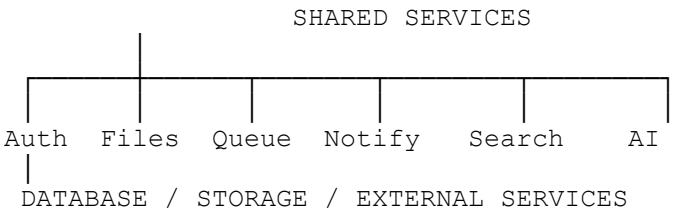
Layer	Recommended Technology
Tables	DataTables
Advanced Selects	Select2
Authentication	Laravel authentication framework
Authorization	RBAC / Policies / Permissions
Cache	Redis
Queues	Redis-backed Laravel Queues
APIs	REST API
Storage	Local/Object Storage abstraction
Maps	Configurable GIS provider
Notifications	Email + SMS + WhatsApp + Push
AI	Provider-agnostic AI service layer
Web Server	Nginx
OS	Linux server environment
Version Control	Git
CI/CD	Automated deployment pipeline
Mobile	API-driven mobile applications

The architecture must avoid tightly coupling business logic to any single external provider.

CORE APPLICATION ARCHITECTURE

RREP should use a layered modular architecture:





MODULAR DEVELOPMENT STANDARD

Each business module shall have:

```
Module
├── Models
├── Controllers
├── Services
├── Requests
├── Policies
├── Events
├── Listeners
├── Jobs
├── Notifications
├── Resources
├── Routes
├── Views
├── Tests
└── Documentation
```

Business logic must not be placed directly inside controllers.

Controllers should primarily:

1. Receive requests.
2. Validate input.
3. Call application services.
4. Return responses.

DATABASE PRINCIPLES

The database shall use a normalized relational structure.

Every major business entity should have:

- Primary key
- UUID/public identifier where appropriate
- Created timestamp
- Updated timestamp

- Created-by reference where required
- Updated-by reference where required
- Status
- Soft-delete capability where appropriate

The system shall maintain clear relationships between:

```
Organization
    ↓
Branches
    ↓
Users
    ↓
Properties
    ↓
Listings
    ↓
Customers
    ↓
Leads
    ↓
Reservations
    ↓
Transactions
    ↓
Payments
```

Additional relationships shall support:

```
Land
 → Parcels
 → Survey
 → GIS
 → Ownership
 → Development
 → Property
```

and:

```
Property
 → Listing
 → Marketing
 → Lead
 → Reservation
 → Sale / Lease
 → Payment
 → Reporting
```

API STANDARD

All major business functions shall be accessible through versioned APIs.

Example:

```
/api/v1/properties
/api/v1/customers
/api/v1/leads
/api/v1/reservations
/api/v1/payments
/api/v1/surveys
/api/v1/reports
```

APIs shall support:

- Authentication
- Authorization
- Validation
- Pagination
- Filtering
- Sorting
- Search
- Standardized responses
- Standardized errors
- Rate limiting
- Logging

AUTHENTICATION & AUTHORIZATION

The platform shall implement:

- Secure authentication
- Password policies
- Password reset
- Email verification
- Session management
- Optional two-factor authentication
- Role-based access control
- Permission-based authorization
- Branch restrictions
- Module restrictions
- Feature restrictions

Example:

```
Organization Administrator
        ↓
Branch Manager
        ↓
Department Manager
        ↓
Employee
```

Permissions must be checked server-side regardless of frontend visibility.

FEATURE LICENSING

The platform shall have a centralized feature/license system.

Example:

```
LICENSE
│
├── Property Management = ENABLED
├── CRM = ENABLED
├── Finance = ENABLED
├── Survey = DISABLED
├── AI = ENABLED
├── Advanced Analytics = DISABLED
└── API Access = ENABLED
```

Features must be controlled through configuration rather than hard-coded customer-specific modifications.

This is critical to keeping the product reusable.

SHARED SERVICES

The following shall be implemented as reusable platform services:

Authentication Service

Identity and access.

Notification Service

Email, SMS, WhatsApp and push notifications.

File Service

Documents, images and attachments.

Audit Service

Activity and change tracking.

Workflow Service

Approvals and automated processes.

Search Service

Global and module-specific search.

Reporting Service

Reusable report generation.

Queue Service

Asynchronous processing.

AI Service

Provider-independent AI functionality.

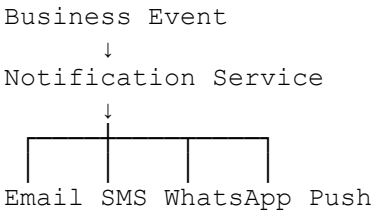
Integration Service

External APIs, webhooks and third-party systems.

NOTIFICATION ARCHITECTURE

Notifications shall use a unified abstraction.

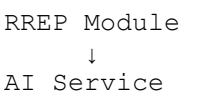
Example:



The business module should not contain provider-specific notification logic.

AI ARCHITECTURE

AI should be implemented as a service layer.



```
     ↓
AI Provider Adapter
     ↓
Selected AI Provider
```

This allows RehoSpace to change AI providers without rewriting business modules.

AI shall support:

- Chat
- Natural-language search
- Summarization
- Content generation
- OCR
- Recommendations
- Forecasting
- Anomaly detection
- Intelligent automation

AI must respect existing authorization rules.

GIS ARCHITECTURE

GIS functionality shall be abstracted from the business modules.

The platform shall support:

- Coordinates
- Boundaries
- Parcels
- Properties
- Maps
- Geolocation
- Spatial search
- Map visualization

The GIS provider should be configurable.

SECURITY BASELINE

The system shall follow secure development practices based on recognized security standards, including OWASP principles.

Minimum requirements:

- HTTPS
- Secure password hashing
- CSRF protection
- Input validation
- Output escaping
- SQL injection protection
- XSS protection
- Authorization checks
- Rate limiting
- Secure file uploads
- Audit logging
- Encryption of sensitive information
- Secure secrets management

FILE & DOCUMENT MANAGEMENT

All uploaded files shall be handled through a centralized storage service.

Supported content includes:

- Property images
- Contracts
- Identification documents
- Survey documents
- Receipts
- Invoices
- Reports
- Marketing materials

Files must have:

- Ownership
- Metadata
- Access permissions
- Storage path
- MIME type
- Size
- Upload history
- Optional version history

QUEUES & BACKGROUND PROCESSING

Long-running operations shall not block normal HTTP requests.

Queue candidates include:

- Sending bulk SMS
- Sending emails
- Generating large reports
- Document processing
- AI processing
- Image processing
- Data imports
- Data exports
- Scheduled notifications
- Automated workflows

SEARCH

RREP shall provide global search across permitted business records.

Search may include:

- Properties
- Customers
- Owners
- Leads
- Transactions
- Documents
- Payments
- Parcels

Search results must always respect authorization.

REPORTING & ANALYTICS

Reports shall support:

- Filters
- Date ranges
- Branch
- Department
- User
- Property
- Transaction
- Export

- PDF
- Excel/CSV where appropriate

Analytics dashboards should consume reusable reporting services rather than duplicate business logic.

DEVELOPMENT STANDARDS

Developers shall follow:

- Git-based version control
- Code reviews
- Consistent naming
- Modular code
- Automated tests
- Secure coding
- Database migrations
- Seeders/factories
- API documentation
- Technical documentation

No production feature should be merged without appropriate testing.

TESTING STRATEGY

Testing shall occur at multiple levels:

Unit Testing

Individual services and business rules.

Feature Testing

Complete application functionality.

API Testing

Request, response, authentication and authorization.

Integration Testing

External services and internal module communication.

Security Testing

Authentication, authorization, input validation and vulnerabilities.

Performance Testing

Large datasets, concurrent users and high-volume requests.

User Acceptance Testing

Business validation before release.

DEVELOPMENT ENVIRONMENTS

At minimum:

```
LOCAL
  ↓
DEVELOPMENT
  ↓
STAGING
  ↓
PRODUCTION
```

Production data must never be casually used in development environments.

VERSION CONTROL

Recommended branch structure:

```
main
develop
feature/*
bugfix/*
hotfix/*
release/*
```

Every feature should be developed in its own branch and reviewed before merging.

DEPLOYMENT

Production deployment should be automated as much as practical.

Deployment should include:

1. Code update
2. Dependency installation
3. Database migration
4. Cache management
5. Asset deployment
6. Queue restart
7. Health verification

Every deployment must be reversible where practical.

BACKUP

The production environment shall maintain backups of:

- Database
- Uploaded files
- Configuration
- Critical application data

Backups shall be:

- Automated
- Monitored
- Retained according to policy
- Tested through periodic restoration

A backup that has never been restored successfully shall not be considered reliable.

MONITORING

Production monitoring should cover:

- CPU
- RAM
- Storage
- Database
- Queue health
- API response times
- Failed jobs
- Application errors
- Authentication anomalies
- External service failures

Critical failures should trigger notifications to administrators.

DISASTER RECOVERY

The system shall define:

- Recovery Point Objective (RPO)
- Recovery Time Objective (RTO)
- Backup frequency
- Recovery procedures
- Emergency contacts
- Failover procedures

Disaster recovery procedures shall be periodically tested.

IMPLEMENTATION STRATEGY

Development should proceed in controlled phases.

Phase 1 — Core Platform

Build:

- Authentication
- Organization
- Branches
- Users
- Roles
- Permissions
- Configuration
- Dashboard
- Audit foundation

Phase 2 — Property Foundation

Build:

- Properties
- Land
- Parcels
- Units
- Property documents
- Property media

- Locations

Phase 3 — CRM & Sales

Build:

- Customers
- Leads
- Agents
- Sales pipeline
- Appointments
- Reservations
- Sales transactions

Phase 4 — Finance

Build:

- Invoices
- Payments
- Installments
- Expenses
- Commissions
- Financial reports

Phase 5 — Rental & Property Management

Build:

- Tenants
- Leases
- Rent
- Maintenance
- Occupancy
- Owner statements

Phase 6 — Survey & GIS

Build:

- Survey projects
- Parcels
- Coordinates
- Maps
- GIS workflows
- Survey documentation

Phase 7 — Marketing & Communication

Build:

- Campaigns
- Templates
- SMS
- Email
- WhatsApp
- Push notifications
- Customer communication

Phase 8 — Portals & Mobile

Build:

- Customer portal
- Owner portal
- Agent portal
- Mobile applications

Phase 9 — Analytics

Build:

- Dashboards
- KPIs
- Reports
- Business intelligence

Phase 10 — AI

Build:

- AI assistant
- Intelligent search
- Recommendations
- Document intelligence
- Predictive analytics
- Smart automation

Phase 11 — Enterprise Hardening

Complete:

- Security testing
- Performance optimization

- Disaster recovery
- Monitoring
- Licensing
- Deployment automation
- Documentation

IMPLEMENTATION RULE

The team shall **not attempt to build all modules simultaneously**.

Every module must progress through:

```
Requirements
    ↓
Database
    ↓
Backend Services
    ↓
APIs
    ↓
Frontend
    ↓
Integration
    ↓
Testing
    ↓
Documentation
    ↓
Acceptance
```

Only after acceptance should the module be considered complete.

DEFINITION OF DONE

A feature is considered complete only when:

- Business requirements are implemented.
- Database changes are complete.
- Backend logic is implemented.
- Authorization is implemented.
- UI is implemented.
- API is implemented where required.
- Validation is implemented.
- Error handling is implemented.
- Audit requirements are implemented.
- Notifications are implemented where required.
- Automated tests exist.

- Security requirements are satisfied.
- Documentation is updated.
- The feature passes acceptance testing.

UI/UX DESIGN SYSTEM

The interface shall follow a consistent enterprise design system.

The design language should prioritize:

- Clarity
- Speed
- Consistency
- Accessibility
- Responsiveness
- Professional appearance

Core UI components shall be standardized:

- Navigation
- Sidebar
- Header
- Cards
- Tables
- Forms
- Modals
- Buttons
- Alerts
- Badges
- Tabs
- Filters
- Search
- Dashboards
- Charts

RESPONSIVE DESIGN

The platform shall support:

- Desktop
- Laptop
- Tablet
- Mobile

Business-critical operations should remain usable on smaller screens.

ACCESSIBILITY

The UI shall consider:

- Keyboard navigation
- Appropriate contrast
- Clear labels
- Form accessibility
- Screen-reader compatibility
- Meaningful error messages
- Responsive text

CUSTOMER EXPERIENCE

Customer-facing interfaces shall be significantly simpler than administrative interfaces.

Customer users should be able to:

- Search properties
- View details
- Save properties
- Contact agents
- Book appointments
- Reserve properties
- Make payments
- Upload documents
- Track transactions
- Receive notifications

without requiring knowledge of internal organizational workflows.

ADMINISTRATIVE EXPERIENCE

Administrative interfaces shall prioritize:

- Information density
- Fast navigation
- Bulk actions
- Advanced filtering

- Reporting
- Configuration
- Permission management
- Auditability

OPERATIONS

Every customer deployment shall have a documented operational configuration covering:

- Organization information
- Branding
- Users
- Roles
- Modules
- License
- Payment configuration
- Notification providers
- Email
- SMS
- WhatsApp
- Maps
- AI provider
- Storage
- Backup
- System preferences

LICENSING IMPLEMENTATION

The application shall distinguish between:

```
CORE PLATFORM
+
LICENSE
+
ENABLED MODULES
+
ENABLED FEATURES
+
OPTIONAL ADD-ONS
```

The application should gracefully disable unavailable functionality rather than exposing broken screens.

For example:

```
if module_enabled('survey'):
```

```
    display Survey functionality
else:
    hide Survey functionality
```

License enforcement must occur server-side.

CUSTOMER DEPLOYMENT MODEL

Each customer installation should have:

```
Customer Server
│
├── RREP Application
├── Customer Database
├── Customer Files
├── Customer Configuration
├── Customer License
├── Customer Integrations
└── Customer Backups
```

RehoSpace maintains the master product codebase.

Customer-specific configuration should be stored separately from core product code.

PRODUCT UPDATE STRATEGY

Product updates shall be versioned.

Example:

```
RREP 1.0
RREP 1.1
RREP 1.2
RREP 2.0
```

Updates should distinguish between:

- Bug fixes
- Security patches
- Minor features
- Major features
- Database changes
- Breaking changes

MASTER PRODUCT GOVERNANCE

RehoSpace shall remain responsible for:

- Product roadmap
- Architecture
- Source code
- Security standards
- Release management
- Licensing
- Module definitions
- Product documentation
- Core integrations
- AI strategy

Customer organizations shall not modify the core product source code as part of normal implementation.

Where customer-specific requirements arise, RehoSpace should preferably solve them through:

1. Configuration
2. Existing module capabilities
3. Feature flags
4. Integration
5. New reusable product capability

rather than customer-specific forks.

FINAL DEVELOPMENT PRINCIPLE

The development team must always ask:

"Can this capability be reused by the next RREP customer?"

If the answer is no, the implementation should be reconsidered unless the requirement is genuinely customer-specific and cannot reasonably become a configurable product capability.

This principle is what transforms RREP from a one-off real estate project into a **reusable commercial software product**.